An implementation of NVIDIA's Agentic Variation Operators (AVO) for Autonomous Evolutionary Search

Marcos Cannabrava
https://github.com/marcoscannabrava/avocode

October 2026

Abstract

Agentic Variation Operators (AVO) replace the fixed mutation and crossover steps of evolutionary search with an autonomous coding agent: $\text{Vary}(\mathcal{P}_t) = \text{Agent}(\mathcal{P}_t, \mathcal{K}, \mathbf{f})$. NVIDIA used AVO to evolve attention kernels on B200 GPUs for seven days, and the result beat cuDNN and FlashAttention-4. The paper did not release its harness, its agent or its supervisor.

This article describes *avocode*, an open, agent-agnostic implementation of the AVO harness. avocode maps each formal object of AVO to a concrete artifact in a git repository. The scoring function $\mathbf{f}$ is an executable that prints one JSON line. The lineage $\mathcal{P}_t$ is a chain of git commits with score trailers. The knowledge base $\mathcal{K}$ is a hybrid-search index over documents *and* over the lineage itself. The agent is any of three coding agents (pi, Claude Code or Codex), driven by five portable skill files. We make precise what the paper leaves open. The commit rule is weak Pareto dominance over a score vector, with strict gain on at least one configuration and a relative noise floor. The supervisor is a deterministic detector of two signals, *stall* and *thrash*, that emits a directive citing prior versions, recorded dead ends and unread documents. A fan-out operator runs N probes in disposable git worktrees as a bounded first step toward the population search that the paper leaves as future work.

On a fuzzy string search benchmark, two runs of the loop with Claude Code (65 minutes in total) committed seven versions and cut the median search time from 1810 ms to 0.231 ms, a 7837× speedup, with no regression committed. One of the seven steps, a pigeonhole partition index, is absent from the hand-written reference solutions. We also report what this evidence does not show. The supervisor never fired in a real run, all results come from one benchmark and one agent, and the integrity of $\mathbf{f}$ rests partly on convention rather than enforcement.

1 Introduction

Evolutionary search improves a solution by repeating two steps. A *variation* operator makes new candidates from old ones, and a *selection* step keeps the better ones [7]. For decades the variation operator was hand-designed: bit flips, subtree swaps and crossover [42]. Large language models (LLMs) changed this. In ELM [32], FunSearch [47], EvoPrompting [9] and AlphaEvolve [41], an LLM writes the new candidate. But a fixed framework still samples the parents, runs the evaluation and manages the population. The LLM fills one slot in a pipeline:

$$\text{Vary}(\mathcal{P}_t) = \text{Generate}(\text{Sample}(\mathcal{P}_t)). \tag{1}$$

The AVO paper [10] argues that this slot is too small. An LLM confined to Generate "produces a single output per invocation, with no ability to proactively consult reference materials, test its

changes, interpret feedback, or revise its approach before committing a candidate" [10, §1]. AVO gives the whole variation step to an autonomous coding agent:

$$\mathrm{Vary}(\mathcal{P}_t) = \mathrm{Agent}(\mathcal{P}_t, \mathcal{K}, \mathbf{f}), \tag{2}$$

where $\mathcal{P}_t$ is the lineage of solutions and their scores, $\mathcal{K}$ is a domain knowledge base and $\mathbf{f}$ is the scoring function. The agent decides what to read, what to edit and when to test. NVIDIA ran this operator for seven days on multi-head attention for Blackwell B200 GPUs. It committed 40 versions and reached up to 1668 TFLOPS, which is up to 3.5% faster than cuDNN and up to 10.5% faster than FlashAttention-4 on the tested configurations.

The paper's result is about kernels. Its *harness* is not. Equation (2) needs only four things: a way to measure a candidate, a record of the versions that survived, a place to look things up, and an agent. None of them is specific to GPUs. But the paper releases no code, names no model and gives no supervisor thresholds. It leaves the acceptance rule for a vector score as the phrase "matches or improves". A reader who wants to apply AVO to a different problem has to rebuild the harness from the prose.

This article describes such a rebuild. *avocode* is a command-line tool (`avo`) and a set of agent skills that turn any git repository with a measurable score into an AVO loop. Its thesis is that the harness generalizes past attention kernels to any repository with a strict, cheap scorer. The design follows one rule throughout: the agent proposes, and the measurement decides.

Contributions.

1. **A concrete mapping of AVO to a repository** (Section 4). $\mathbf{f}$ becomes an executable with a one-line JSON contract, $\mathcal{P}_t$ becomes git commits with score trailers, and $\mathcal{K}$ becomes a hybrid-search index whose corpus includes the lineage itself.
2. **A precise commit rule** (Section 6). We define acceptance as weak Pareto dominance over the score vector with strict gain on at least one configuration and a symmetric relative noise floor. We state the invariants this rule gives and the ways it resists gaming.
3. **An external, deterministic supervisor** (Section 7.3). It detects two signals, stall and thrash, and emits a directive that *cites* prior versions, recorded dead ends and documents in $\mathcal{K}$ that no version has used.
4. **Agent-agnostic delivery** (Section 7.5). The same five skill files drive pi, Claude Code and Codex. Every capability is reachable from a shell.
5. **A bounded fan-out operator** (Section 7.4). `avo fan` runs N probes in disposable git worktrees on a small model. It is a first, local step toward the population search that the paper defers.
6. **An empirical case study and an honest list of limits** (Sections 8 and 10). Two runs on a fuzzy search benchmark reach a 7837× speedup with no committed regression. We also show where the evidence is thin.

Outline. Section 2 places AVO among related work. Section 3 summarizes the AVO paper. Sections 4 to 7 describe avocode, from the system view down to the commit rule and the loop. Section 8 reports results. Section 9 compares avocode with the paper, and Section 10 lists limitations and future work.

2 Background and related work

2.1 Classical evolutionary computation

Genetic algorithms [20] and genetic programming [25] keep a population, vary it with blind syntactic operators, and select by fitness. Genetic programming is the direct ancestor of "evolve code against an executable fitness function". Its long-standing problems include operator locality, where a small syntactic change causes a large semantic change, and poor scaling to large programs [42]. The AVO paper cites this survey to motivate operators that understand code.

The closest classical model of avocode is the $(1 + 1)$ evolutionary algorithm [7, 16]. It keeps one parent, mutates it, and keeps the child if it is at least as good. avocode differs in two ways. It accepts by dominance over a score vector rather than by a scalar $\geq$, and it refuses ties, so it allows no neutral drift. With `avo fan`, the scheme becomes $(1 + \lambda)$: λ probes run in parallel, and at most one is promoted. A single elitist lineage is a hill climber, and its known failure is stagnation at a local optimum [48]. Novelty search [30] and quality-diversity archives such as MAP-Elites [40] answer this by keeping diverse stepping stones. AVO and avocode drop the archive on purpose and rely on the agent, which reads the whole lineage, to supply diversity. Section 10 returns to this cost.

2.2 LLMs as variation operators

ELM [32] first used a code LLM as the mutation operator inside MAP-Elites. Language model crossover [39] puts several parents in one prompt. EvoPrompting [9], EvoPrompt [19] and Promptbreeder [17] evolve architectures and prompts in the same way. OPRO [57] is the simplest "lineage in context" optimizer: past solutions and their scores go into the prompt. FunSearch [47] paired a code LLM with an evaluator and an island-based program database, and found new cap-set constructions. AlphaEvolve [41] scaled this to whole files, an LLM ensemble and asynchronous evaluators, with results in matrix multiplication, scheduling and kernels [18]. ReEvo [60] adds a reflection stage, and LoongFlow [52] replaces random mutation with a plan–execute–summarize cycle.

Open re-implementations exist. OpenEvolve [50] rebuilds AlphaEvolve's pipeline. ShinkaEvolve [27] adds novelty rejection and a bandit over models for sample efficiency, and CodeEvolve [6] uses island search over a CVT-MAP-Elites archive. These all implement the fixed Generate $\circ$ Sample pipeline that AVO removes. The avocode design notes reject them as a base for this reason. Another line changes the operator's weights rather than its context, through test-time reinforcement learning [56, 63]. avocode trains nothing; it only shapes the context through the lineage, memory and knowledge.

2.3 Coding agents and autonomous research loops

AVO depends on agents that can work at repository scale. ReAct [59] interleaves reasoning and tool use. SWE-bench [23] measures real issue resolution, and SWE-agent [58] shows that the agent–computer interface matters: a few well-designed commands raise scores a lot. avocode is an interface of this kind for evolution. It gives the agent a few commands with JSON output, and skills that say when to use each one. The operators it drives are pi [65], Claude Code [4] and Codex [44]; OpenHands [54] could be added with one template.

Reflexion [51] stores verbal reflections after a failure, and Voyager [53] keeps a library of executable skills. avocode makes both explicit and durable: dead ends and insights go to memory that a new session reads, and skills are files in the repository. AIDE [22] searches a tree of scripts. The AI Scientist [35, 55] automates a full research pipeline. The Darwin Gödel Machine [67] evolves

the agent itself, as do STOP, Gödel Agent and ADAS [21, 62, 66]. These systems improve the operator; AVO and avocode improve a target with a fixed operator.

The closest relatives of avocode appeared in 2026. Karpathy's *autoresearch* [24] is a minimal loop: an agent edits one training script, runs a five-minute job, and keeps the change only if validation loss improves. avocode generalizes it with a scorer contract, a score vector with a noise floor, recorded refusals, searchable memory, parallel probes and a supervisor. Bilevel Autoresearch [46] shows that improving the harness can matter as much as the model. Agora [68] also uses git as the store of results, as a directed acyclic graph shared by many workers. SEDIMA [1] adds a memory of insights to evolutionary agents because memoryless search "rediscovers the same improvements and re-encounters the same dead ends". Chun et al. [12] list the failure modes of such agents: they fix the same bugs again and again and do not explore. avocode's dead-end memory, thrash detector and fan-out target exactly these modes.

2.4 LLMs for performance engineering

The FlashAttention series [14, 15, 49, 64] and cuDNN [11] are AVO's baselines. KernelBench [45] measures whether LLM kernels are both correct and fast, which has the same shape as avocode's $\mathbf{f}$: a correctness gate, then a metric. Kevin [8], CUDA-L1 [33] and CUDA Agent [13] train the refinement loop into the model; AVO and avocode put it into the harness. SOL-ExecBench [34] scores against speed-of-light bounds, which a scorer could report as an absolute scale.

2.5 Evaluation integrity

An optimizer under strong pressure games its proxy [3, 26, 37]. Evolutionary search has exploited bugs in fitness functions for decades [31]. With agents, the problem is faster and deliberate. METR reports frontier models that read the grader's answer or patch the timer [38]; in one case a model took the reference output from the call stack and disabled CUDA synchronization to fake a fast kernel. Sakana's AI CUDA Engineer [28] withdrew speedups that came from an evaluation exploit, and robust-kbench [29] closes such loopholes. ImpossibleBench [69] measures how often agents exploit tests. The AVO paper does not discuss this threat in detail. avocode treats it as a first-class design concern (Section 6.3).

Table 1 places avocode among these systems.

3 AVO in brief

This section summarizes the AVO paper [10] (arXiv v1, March 2026). We keep its notation. Where the paper is silent, we say so, because those gaps are where an implementation must make choices.

3.1 Formulation

Generic evolutionary search keeps a population $\mathcal{P} = \{(x_i, \mathbf{f}(x_i))\}$ of solutions and their scores, and repeats

$$\mathcal{P}_{t+1} = \mathrm{Update}(\mathcal{P}_t, (x_{t+1}, \mathbf{f}(x_{t+1}))), \qquad x_{t+1} = \mathrm{Vary}(\mathcal{P}_t). \tag{3}$$

Prior LLM methods split Vary into Generate $\circ$ Sample (Equation (1)). The paper notes that "the LLM only participates in Generate: the sampling strategy, evaluation protocol, population management, and the order of operations are all determined by the framework" [10, §2.1]. AVO replaces the whole operator with one agent run (Equation (2)). The agent "subsumes Sample, Generate, and evaluation into a single autonomous loop".

Table 1: Where avocode sits. "Pipeline" denotes FunSearch, AlphaEvolve, OpenEvolve and ShinkaEvolve.

Axis	Pipeline systems	AVO	avocode
Operator	one LLM call in a fixed pipeline	one internal agent loop	any coding agent (pi, Claude Code, Codex)
Selection	population, islands, MAP-Elites	single lineage, "matches or improves"	single lineage; vector dominance with a noise floor
Diversity	archive, islands	agent reads the lineage	agent, `avo fan` probes, supervisor
Memory	program database	conversation history	git lineage, searchable $\mathcal{K}$, dead-end memory
Integrity	sandboxed evaluator	not discussed	re-score at commit, no measuring less, hash gates, external audit
Release	open (OpenEvolve, ShinkaEvolve)	none	open source

The objects are:

- $\mathcal{P}_t = \{(x_1, \mathbf{f}(x_1)), \ldots, (x_t, \mathbf{f}(x_t))\}$, "the full lineage of solutions and their scores". In the experiments, each x_i is a CUDA kernel with inline PTX.
- $\mathbf{f}(x_i) = (f_1(x_i), \ldots, f_n(x_i))$, a vector with one component per test configuration. A candidate that fails the correctness check gets $f_j(x_i) = 0$ for all j, "regardless of throughput".
- $\mathcal{K}$, a knowledge base: CUDA guides, the PTX ISA, Blackwell specifications, and existing kernels including the FlashAttention-4 source.

3.2 One variation step

A step that produces x_{t+1} is an autonomous agent loop with "numerous internal actions". The agent often reads several prior versions in $\mathcal{P}_t$, compares their profiles, and consults $\mathcal{K}$ before it edits. It then calls $\mathbf{f}$. If the candidate fails or does not improve, the agent diagnoses and revises. The paper calls this the *edit–evaluate–diagnose* cycle. The rule for keeping a result is stated once:

> "we persist a new committed version only when it passes correctness checks and matches or improves the benchmark score relative to the best committed version so far; unsuccessful intermediate attempts remain part of the agent's internal search trajectory but are not added to the committed lineage." [10, §3.2]

Read literally, this is an elitist, monotone update: $\mathcal{P}_{t+1} = \mathcal{P}_t \cup \{(x', \mathbf{f}(x'))\}$ if x' is correct and scores at least as well as the best, and $\mathcal{P}_{t+1} = \mathcal{P}_t$ otherwise. Nothing is pruned. The paper does not say how "matches or improves" applies to a *vector*, or what tolerance counts as a match. Its trajectory plots use the geometric mean across configurations, which suggests a scalar reduction, but the text does not commit to one.

3.3 Continuous evolution and the supervisor

The paper runs AVO as a single lineage from a seed x_0, "leaving population-level branching and archive management to future extensions" [10, §3.3]. Each committed version is a git commit with its score. Two failure modes appear in long runs: the agent can *stall* when it exhausts a line of

Table 2: Headline results of the AVO paper [10, §4]. Ranges are over the four sequence lengths. TFLOPS values are the paper's bar labels.

Workload	Peak AVO TFLOPS	vs. cuDNN	vs. FlashAttention-4
MHA, causal	1637	+0.4% to +3.5%	+5.0% to +10.5%
MHA, non-causal	1668	+1.8% to +2.4% (16K, 32K)	within noise at 4K, 8K
GQA, causal	1647	up to +7.0%	up to +9.3%
GQA, non-causal	1679	up to +6.0%	up to +4.5%
Run: 7 days, 40 committed versions, >500 directions explored, 1 lineage.			

exploration, or it can enter "unproductive cycles of edits that repeatedly fail to improve scores". A self-supervision mechanism detects both. When triggered, it "reviews the overall evolutionary trajectory and steers the search toward several candidate optimization directions". Figure 2 of the paper draws this as a separate *AVO Supervisor Agent*. The paper gives no thresholds, no trigger counts, and no mechanism for how the directions reach the main agent.

The agent itself is "an internally-developed general-purpose coding agent powered by frontier LLMs" with code editing, shell, file navigation and documentation retrieval. Its memory is its conversation history. The models are not named, and no token or cost figures are given.

3.4 Reported results

Table 2 summarizes the results on B200 (forward prefill, head dimension 128, BF16, sequence lengths 4K–32K at a fixed 32K total tokens). The seven-day run produced 40 versions and explored "over 500 candidate optimization directions". Gains came in discrete jumps, for example branchless accumulator rescaling (+8.1% non-causal at v20) and register rebalancing across warp groups (+2.1% at v33). The paper uses these to argue that the agent performs "genuine hardware-level reasoning rather than superficial code transformations". Adapting the final kernel to grouped-query attention (GQA) took 30 more minutes of autonomous work.

3.5 What the paper leaves open

For an implementer, the important gaps are:

1. **No code or harness.** No repository, prompts, or agent configuration.
2. **An underspecified acceptance rule.** "Matches or improves" on a vector, with no noise band, although the authors measured each benchmark 10 times for mean and deviation.
3. **An unspecified supervisor.** No detection rule, threshold, or delivery mechanism.
4. **Trajectory is lost.** Failed attempts live only in the agent's context, so a new session cannot learn from them.
5. **No population.** Branching and archives are future work.
6. **No ablations of the harness.** Neither $\mathcal{K}$ nor the supervisor is ablated, and AVO is not compared with a Generate $\circ$ Sample baseline on the same budget.

avocode gives a concrete answer to items 1–4 and a partial answer to item 5. It does not address item 6. Section 9 returns to each.

Table 3: How avocode implements each object of AVO.

Symbol	Meaning	avocode artifact	Module
$\mathbf{f}$	scoring function	executable `.avo/score` printing one JSON line	`score.ts`
$\succ$	order on $\mathbf{f}$	vector comparator, noise floor, reduction	`compare.ts`
$\mathcal{P}_t$	lineage	git commits with `Avo-Version`/`Avo-Score` trailers, `git notes`, `lineage/vNNN.md`	`lineage.ts`
$\mathcal{K}$	knowledge base	`knowledge/` + `lineage/`, hybrid search; web ingest with provenance	`knowledge.ts`
–	agent memory	beads issue graph, or `lineage/memory.jsonl`	`mem.ts`
Agent	variation operator	pi, Claude Code or Codex, driven by five skills	`agents.ts`
–	supervisor	stall/thrash detector with a citing directive	`supervise.ts`
–	trajectory	`.avo/attempts.jsonl`, `.avo/runs/`, `.avo/worktrees/` (never committed)	`score.ts`

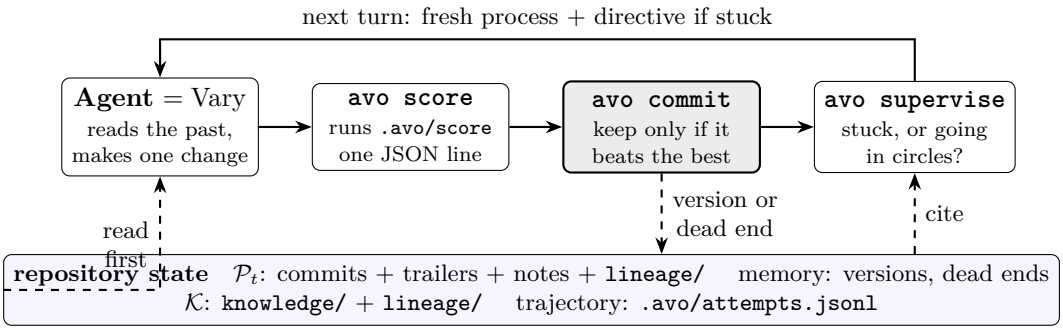


Figure 1: The avocode loop. The agent proposes; the score and the commit rule decide. Every outcome, kept or refused, is written to the repository that the next turn reads.

4 System overview

avocode extracts the AVO harness and makes it independent of the agent and of the domain. Table 3 gives the mapping from the paper's objects to artifacts in a repository. Figure 1 shows the loop.

4.1 Three layers

avocode has three layers, and only the first is required.

CLI. `bin/avo` is a bash shim that runs `src/main.ts` through `tsx`. There is no build step. The CLI is the only interface that must exist. An invariant of the design is that every capability is reachable from `bash` before it gets any other binding.

Skills. Five Markdown files in `.agents/skills/` follow the Agent Skills format [2]. They teach an agent the loop: `avo-vary` (one variation step), `avo-score` (the $\mathbf{f}$ contract), `avo-lineage` (reading $\mathcal{P}_t$), `avo-knowledge` ($\mathcal{K}$) and `avo-fanout` (parallel probes). The project calls the skills "the product": the CLI enforces, and the skills teach.

Bindings. Native tools and an in-session supervisor for the pi agent (Section 7.5). These add convenience, never capability.

Table 4: Source modules of avocode (TypeScript, about 9,100 lines in `src/`).

File	Owns	Lines
`cli.ts`	subcommand dispatch, flags, exit codes	162
`score.ts`	**f**: run `.avo/score`, validate, normalize into an attempt	644
`config.ts`	`.avo/config.json`: reduction, noise floor, configs, thresholds	172
`compare.ts`	the commit rule's comparator over the score vector	235
`lineage.ts`	$\mathcal{P}_t$: `avo commit`, `avo lineage`, `avo best`	805
`mem.ts`	memory: beads with a JSONL fallback	447
`knowledge.ts`	$\mathcal{K}$: qmd collections, local fallback, ingest with provenance	985
`websearch.ts`	$\mathcal{K}$ from the web: three backends behind one seam	324
`agents.ts`	headless agent templates; one turn as `fan` and `run` see it	436
`fan.ts`	`avo fan`: worktrees, probes, guards, promote, resume	1065
`supervise.ts`	`avo supervise`: stall/thrash detector and directive	550
`run.ts`	`avo run`: the continuous loop and its manifest	823
`init.ts`, `install.ts`, `doctor.ts`, `skills.ts`	scaffolding, agent wiring, dependency checks, skill validation	927

4.2 One file, one contract

Each source file owns one contract (Table 4). The pi extension imports the same files as the CLI, so the commit rule, the detector and the recursion guards each exist once. Every external effect goes through an injectable seam: an `Io` sink for output, a `Runner` for processes and a `Fetcher` for the network. This is why the 441 unit tests run without a terminal, a scorer, git or a network.

4.3 Design invariants

The design document lists nine invariants. They explain most choices in the rest of the article.

1. `avo commit` is the only writer of the committed lineage.
2. A failing **f** never yields a commit.
3. Every subcommand supports `-json`.
4. Degrade, never crash: each optional tool has a named fallback and one warning.
5. Idempotent by construction (`init`, `install`, `commit`).
6. Never leak secrets: report the presence of a key, never its value.
7. Worktrees are disposable; the main branch is not.
8. CLI first: a capability is reachable from `bash` before any binding.
9. The skills are the product.

5 The scoring function f

In AVO, **f** checks correctness against a reference and measures throughput per configuration. In avocode, **f** is any executable at `.avo/score`, run from the repository root. This keeps the harness

domain-free: a benchmark, a test suite, or a game simulator can all be a scorer.

5.1 The contract

The scorer has three call forms:

```
.avo/score                   # score everything; print one JSON line
.avo/score --configs         # optional: list config names, one per line
.avo/score --config <name>   # score one config; print one JSON line
```

It must print one JSON object and should always exit 0. A typical line is:

```
{"ok":true,"correct":true,"primary":0.231,"unit":"ms","higher_is_better":false,
 "scores":{"small":0.158,"large":0.304}}
```

The fields `ok`, `correct`, `primary` (a number or `null`), `unit` and `higher_is_better` are required. The fields `scores` (the vector), `log` and `duration_s` are optional. Unknown keys are allowed, but each one produces a warning, so a typo such as `higherIsBetter` reports both a missing and an unknown field.

The two booleans have different meanings. `ok` says that *the scorer* worked; it is not a verdict on the candidate. `correct` is *the gate*. avocode defines

$$\text{pass}(x) = \text{ok}(x) \wedge \text{correct}(x). \tag{4}$$

If $\neg\text{pass}(x)$, avocode sets `primary` to `null`, whatever the scorer measured. This generalizes the paper's "assign zero score". Zero is the *best* possible value of a lower-is-better metric such as latency, so it cannot mean failure in a domain-free harness.

5.2 Harness errors and results

avocode separates a candidate that fails from a scorer that fails. A spawn failure, a timeout, or output that does not parse is a *harness error* (exit 2). A valid JSON line with $\neg$pass is a *result* (exit 1). Only results teach anything about the candidate, so only results feed memory and the supervisor (Section 6.5). The parser scans standard output backwards for the last line that parses as a JSON object, so build noise before the payload is harmless.

Each scorer run is spawned as the leader of its own process group. On timeout avocode sends `SIGKILL` to the whole group, which also kills benchmark processes that the scorer started. Output is capped at 200,000 characters per stream as a 150,000-character head plus a rolling 50,000-character tail. The tail exists because the payload comes *last*. An earlier head-only cap dropped the final event of agent logs and recorded 44 input tokens for a run that sent 985,039 (Section 8.5).

5.3 Configurations and the score vector

A scorer may report a vector in `scores`, one entry per configuration, which matches the paper's $\mathbf{f} = (f_1, \ldots, f_n)$. With `-parallel`, avocode asks the scorer for its config names and runs `.avo/score -config <name>` for each, with at most $\max(1, \min(8, \mathit{cpus} - 2))$ in parallel. One failing config fails the whole attempt. `primary` is the arithmetic mean of the per-config primaries; it is a headline only. The commit rule uses the vector (Section 6.1).

5.4 The attempt log

Every scorer run appends one record to `.avo/attempts.jsonl`, including the git `HEAD` and a dirty flag. This is the *trajectory*: the attempts that the paper leaves inside the agent's context. In avocode it is a file on disk. It is never committed, but the supervisor reads it (Section 7.3).

5.5 Templates and scorer design

`avo score -init` copies one of three templates: `hyperfine` (mean seconds per config, lower is better, `correct` = a test command passes), `pytest` and `vitest` (pass rate, higher is better). The documentation gives four rules for writing a scorer, and they matter more than any code in the harness:

1. Make `correct` strict and cheap. It is the only thing that rejects a fast wrong answer.
2. Make the benchmark deterministic, or set a noise floor (Section 6.1).
3. Keep it fast. A one-hour scorer allows 24 attempts a day.
4. Measure what you want improved, because that is what the agent will optimize.

Section 8.1 shows two scorers built on these rules, with hash gates over protected files and a full-scale differential check against an independent reference.

6 The lineage $\mathcal{P}_t$, memory and knowledge $\mathcal{K}$

6.1 The commit rule

The commit rule is the selection step of the evolutionary search. In avocode it is the only path by which a version enters $\mathcal{P}_t$ (invariant 1). `avo commit` runs these checks in order, and the first that fires decides:

1. The working tree must differ from `HEAD`, ignoring avocode's own files. Otherwise the result is a *no-op*: nothing is scored and nothing is recorded.
2. The scorer runs *now*. There is no way to commit a number that the agent supplies.
3. A harness error refuses with exit 2.
4. ¬pass refuses with exit 1.
5. The comparator below decides. If it accepts, avocode writes the version.

Notation. Let x_t be the best version and x' the candidate. Let $\mathcal{C}_t$ and $\mathcal{C}'$ be their config sets. Let $\sigma \in \{+1, -1\}$ be $+1$ when higher is better. Define direction-normalized scores $s_c = \sigma f_c(x_t)$ and $s'_c = \sigma' f_c(x')$, so that higher is always better. Let $\varphi \geq 0$ be the noise floor (`floor` in `.avo/config.json`, default 0). The relative change on config c is

$$\rho_c = \begin{cases} 0 & \text{if } s'_c = s_c, \\ \dfrac{s'_c - s_c}{|s_c|} & \text{if } s_c \neq 0, \\ \operatorname{sign}(s'_c - s_c) \cdot \infty & \text{if } s_c = 0,\ s'_c \neq s_c. \end{cases} \tag{5}$$

Candidate vs. best	`small`	`large`	Decision
faster on both	+12%	+9%	kept
tie on one, win on the other	+1%	+5%	kept
win on one, regress on the other	+20%	−4.7%	refused
tie on both	+1%	−2%	refused

Figure 2: The commit rule with a noise floor $\varphi = 3\%$ (the `fuzzysearch` setting). Positive values are improvements after direction normalization. A change inside $\pm\varphi$ is a tie. A win on one config cannot pay for a loss on another, and a tie on every config is not progress. The third row matches the one real refusal in Section 8.

Definition 1 (Commit rule, default `reduce = "dominate"`)**.** A candidate x' becomes x_{t+1} if and only if the tree changed, the scorer ran without a harness error, pass(x') holds, and either $\mathcal{P}_t = \emptyset$ or all of the following hold:

$$\sigma' = \sigma \qquad \text{(the direction is frozen)} \tag{6}$$

$$\mathcal{C}_t \subseteq \mathcal{C}' \qquad \text{(no measuring less)} \tag{7}$$

$$\forall c \in \mathcal{C}_t : \ \rho_c \geq -\varphi \qquad \text{(no config regresses beyond the band)} \tag{8}$$

$$\exists c \in \mathcal{C}_t : \ \rho_c > \varphi \qquad \text{(some config improves beyond the band)} \tag{9}$$

With $\varphi = 0$, conditions (8)–(9) reduce to $\forall c\colon s'_c \geq s_c$ and $\exists c\colon s'_c > s_c$. That is, x' must Pareto-dominate x_t on the incumbent's configs. The project docs call the rule *dominate-or-tie*, where "tie" refers to single configs. A tie on the *whole* vector is refused: "the lineage records progress, not equal-scoring rewrites". Figure 2 shows four cases.

The opt-in mean rule. With `reduce = "mean"`, weights $w_c \geq 0$ (default 1) and $W = \sum_{c \in \mathcal{C}_t} w_c$, let $M(s) = W^{-1} \sum_{c \in \mathcal{C}_t} w_c s_c$. The candidate is accepted if the preconditions (6)–(7) hold, $W > 0$, and the relative change of M exceeds φ. Per-config verdicts are still stored, so a regression that the mean accepts stays visible. Dominance is the default because "a mean lets a large win on one config pay for a regression on another".

Noise. φ is a *relative* band applied in both directions. Noise inside the band can neither commit nor block. A regression outside the band still blocks a gain inside it. avocode has no repeated sampling or confidence interval in the rule; φ is the only noise mechanism, and it should come from measured spread. For `fuzzysearch`, the scripted reference steps showed 0–2.5% interquartile spread, which set $\varphi = 0.03$.

6.2 Properties of the rule

Proposition 1 (Monotone lineage)**.** With $\varphi = 0$ and `reduce = "dominate"`, for every config c and every pair of versions $i < j$ that both measure c, $s_c(x_j) \geq s_c(x_i)$ (as recorded at commit time).

Proof. By (7), a config measured by x_i is measured by every later version. By (8) with $\varphi = 0$, each step does not decrease s_c. Induction on $j - i$. □

Three consequences follow, and the implementation relies on each.

Table 5: Threats to the integrity of **f** and how avocode handles them.

Threat	Mechanism	Kind
Commit a number that was not measured	`avo commit` always re-scores	enforced
Fast but wrong candidate	pass gate before any ranking	enforced
A win on one config pays for a loss	dominance is the default	enforced
Improve by dropping a slow config	(7): `missing-configs`	enforced
Flip the metric direction	(6): `direction-changed`	enforced
Noise commits a non-change	floor φ	enforced, opt-in
A typo in the config disables the gate	malformed config $\Rightarrow$ all defaults	enforced
Hand-written version commit	only trailer-bearing commits count	convention
Edit `.avo/score` to pass	rules in `AGENTS.md` and skills; bench hash gate	convention / bench
Edit `lineage/`	rules; the comparator never reads `lineage/`	convention + design

- **Best = newest.** `avo best` returns the version with the highest number, with no ranking pass. This is correct only because of Proposition 1.
- **Configs only grow.** $\mathcal{C}_t \subseteq \mathcal{C}_{t+1}$. A new config enters with no baseline and is binding for every later version. A scorer edit that drops a slow config is refused as `missing-configs`.
- **The direction is frozen.** A flip is refused with the advice to "start a fresh lineage rather than reinterpreting the old scores".

With $\varphi > 0$ the guarantee weakens to $s'_c \geq s_c - \varphi|s_c|$ per step, so a chain of small in-band losses can drift down on one config over many versions. With the mean rule, only M is monotone. avocode does not re-check these properties; they hold by construction.

Departure from the paper. The paper keeps a version that "matches or improves". avocode requires a strict gain on at least one config. It also never re-measures the incumbent: the best version's vector comes from its stored trailer, while the candidate is measured on today's machine. Section 9 discusses both choices.

6.3 Guarding f against gaming

An optimizer that can edit its own objective will. Table 5 lists the threats that avocode considers and how each one is handled. Some are enforced by code. Others rest on convention: rules in `AGENTS.md` and the skills. The gap matters. `avo commit` stages with `git add -A`, so an edit to `.avo/score` is committed with the candidate. A scorer edit that drops a config or flips the direction is caught. A scorer edit that simply inflates its numbers is not caught by the harness. The benchmarks close part of this gap with a hash gate over protected files inside the scorer and an external audit after the run (Section 8.1).

6.4 Persisting a version

A version exists if and only if a commit reachable from `HEAD` carries a valid `Avo-Version` trailer and a valid `Avo-Score` trailer. Nothing else defines a version. The commit message has three paragraphs:

```
avo v3: 0.408 ms
```

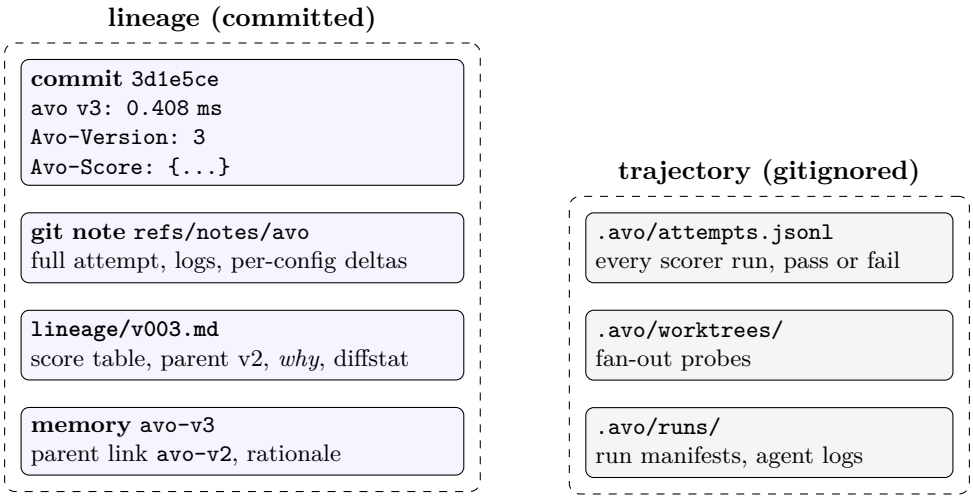


Figure 3: Lineage versus trajectory, for version v3 of the `fuzzysearch` run. The lineage records accepted versions. The trajectory records how they were reached. Memory is committed in the JSONL fallback and lives in the beads database otherwise.

```
partition (pigeonhole) hash index: cut each word into k+1 segments ...

Avo-Version: 3
Avo-Score: {"primary":0.408,"unit":"ms","higher_is_better":false,
            "scores":{"small":0.309,"large":0.507}}
```

Each version has three records with different jobs (Figure 3).

1. **The commit and its trailers**: exactly what the comparator reads back, and nothing more. A damaged note or a deleted Markdown file cannot change a decision.
2. **A git note** under `refs/notes/avo`: the full attempt, logs and per-config deltas.
3. **`lineage/vNNN.md`**: a readable page with the score table, parent, rationale and diffstat. This page is also part of $\mathcal{K}$ (Section 6.7).

The lineage is read with one `git log` using ASCII unit and record separators. Rules for robustness: the last trailer wins (a rebase can prepend an old body), malformed trailers are skipped with a warning, and for duplicate numbers the newest commit wins. Because the lineage is read from the ancestry of `HEAD`, each branch carries its own lineage.

Lineage versus trajectory. avocode keeps *what survived* apart from *how it was reached*. Trajectory paths are gitignored and also unstaged after `git add -A`, in case a repo already tracked them. avocode's own files are also excluded from the dirty check. Without this, the memory record written after v1 would make the next turn see a change the agent never made, score it, refuse it, record the refusal, and dirty the tree again. The project calls this class of bug *self-perturbation*; it appeared in three separate development slices.

6.5 What happens on refusal

A refused candidate leaves no trace in git. It is recorded in three other places.

1. **Trajectory**: the attempt is already in `.avo/attempts.jsonl`.
2. **Memory**: a dead-end record keyed by content, $key = \text{sha256}(reason \parallel vector)_{[0:8]}$, so a repeat of the same dead end updates one record instead of adding a new one.
3. **Output**: the reason names the configs and deltas, for example *"'small' regressed (worst: small* -4.74%*) (floor* $\pm 3.00\%$*)"*.

Harness errors are *not* recorded as dead ends, because "a harness error taught us nothing about the candidate". The refused candidate stays in the working tree, so the agent can keep working on it. The skill tells the agent to add an insight with `avo mem add` when it knows *why* a change failed, because "an unrecorded refusal is one the next session earns again".

6.6 Memory

The paper's agent remembers through its conversation history. avocode moves memory out of the agent and into the repository, which is what makes the operator restartable and agent-agnostic. A memory record has the shape `{ts, kind, key, text, version, parent}`, with four kinds: `insight`, `version`, `failure` and `intervention`. If the beads issue tracker [61] is present, records become beads, and version records are linked parent to child, which mirrors the lineage as a dependency graph. Otherwise they go to `lineage/memory.jsonl`, append-only with last-write-wins per key. The agent reads the same JSON in both cases. `avo mem prime` prints insights, "dead ends (do not re-try these)" and committed versions. Interventions are left out on purpose, so old directives do not echo into new sessions.

6.7 Knowledge $\mathcal{K}$

$\mathcal{K}$ has two Markdown collections: `knowledge/` ("what we were told": docs and papers) and `lineage/` ("what we learned"). Both are indexed by qmd [36], which combines BM25, vector search, LLM query expansion and a local reranker. Because `avo commit` already writes `lineage/vNNN.md`, the question "what did I already try about register pressure?" is the same query as "what do the docs say about it?". This turns the paper's observation that the agent "frequently examines multiple prior implementations in $\mathcal{P}_t$" into a single search over both.

Without a project-local qmd index, a local scan returns the same JSON shape. Its score is the fraction of distinct query terms that match, and it says it is not BM25. Documents can be added from the web with `avo know search -ingest` or `avo know add`. Each added file records its `source`, `title`, `fetched-at` time and `via` backend in front matter, so a claim in $\mathcal{K}$ can be traced and checked later. Re-adding a page with the same body is a no-op; a different body is refused without `-force`.

7 Orchestration

7.1 The variation step as a skill

The `avo-vary` skill tells the agent: "You are the variation operator ... You do *not* decide what gets kept — `avo commit` does." It prescribes one step in a fixed order:

```
avo mem prime                # 1. what past turns learned, incl. dead ends
avo best; avo lineage        # 2. the version to beat, and the whole population
avo know query "<the idea>"  # 3. K: docs and prior lineage entries
# 4. make ONE coherent change
```

Algorithm 1 `avo run`: the outer loop

Require: repo with seed commit x_0; prompt π; agent α; budget T; turn timeout τ; thresholds N (stall), R (thrash)

CheckGuards(π) ▷ depth $<$ cap, sha(π) $\notin$ chain
$d \leftarrow \bot$; $prev \leftarrow \bot$; $idle \leftarrow 0$
for $t = 1, \ldots, T$ **do**
  if `.avo/STOP` exists **then return** STOP-FILE
  end if
  $h_0 \leftarrow$ `HEAD`
  $p \leftarrow$ TurnPrompt($\pi, t, prev, d$) ▷ π verbatim at $t = 1$
  $(ok, summary) \leftarrow$ DriveAgent(α, p, τ) ▷ fresh process: Vary($\mathcal{P}_t$)
  if spawn failed **then return** AGENT-UNAVAILABLE
  end if
  $dec \leftarrow$ Commit($summary$) ▷ Algorithm 2; summary becomes *why*
  $h_1 \leftarrow$ `HEAD`; credit versions in $h_0..h_1$ ▷ agent may commit itself
  $idle \leftarrow (dec = \text{NOOP} \wedge h_1 = h_0)\ ?\ idle + 1 : 0$
  $(sig, d) \leftarrow$ Supervise(N, R) ▷ Algorithm 3
  if $sig \neq \emptyset$ **then** record an `intervention` memory
  end if
  write manifest ▷ after every iteration
  $prev \leftarrow (dec, sig)$
  if $idle \geq 3$ **then return** NO-PROGRESS
  end if
end for
return MAX-ITERS

```
avo score --json              # 5. measure it
avo commit --why "<what changed and why it should help>"
```

Steps 1–3 are the agent's reading of $\mathcal{P}_t$ and $\mathcal{K}$ in Equation (2). The rule "one idea per candidate" makes a refusal attributable to one change. Inside this step the agent is free: it can profile, read code, search the web, or call `avo fan` (Section 7.4). This is the paper's edit–evaluate–diagnose cycle, written as instructions rather than code.

7.2 `avo run`: the continuous loop

`avo run` automates the cycle of Figure 1. Its central choice differs from the paper: *each turn is a fresh agent process.* The paper's agent keeps one conversation for seven days. In avocode, "every iteration is a fresh process, so the turn prompt is the only continuity". All other state lives in the repository: the lineage in git, the trajectory in the attempt log, and memory in beads or JSONL. Algorithm 1 gives the loop.

The turn prompt. The first turn gets the operator's prompt verbatim. Later turns get the prompt plus a footer: the turn number, a one-line outcome of the previous turn (for example *"committed v3 at 0.408 ms"* or *"refused at 0.42 ms — ..."*), the number of attempts since the best version, a reminder that the agent is a fresh process that should read `avo lineage`, `avo best` and `avo mem prime`, and the supervisor's directive if one fired. The directive is *appended* to the task, never substituted for it.

Algorithm 2 COMMIT(*why*): the only writer of $\mathcal{P}_t$

if tree is clean, ignoring harness paths **then return** NOOP
end if
$a \leftarrow \mathbf{f}$(working tree); append a to the attempt log
$b \leftarrow$ newest version in $\mathcal{P}_t$ ▷ best, by Proposition 1
if a has harness errors **then return** ERROR
end if
if $\neg$pass(a) **or** ($b \neq \perp$ **and not** ACCEPT(a, b)) **then** ▷ Definition 1
    record dead end, key = sha256(*reason* $\|$ *vector*)
    return REFUSED
end if
commit with trailers; add git note; write `lineage/vNNN.md`; record version memory
return COMMITTED

Committing. The agent's final message, capped at 2,000 characters, becomes the *why* of the commit. The skill also allows the agent to call `avo commit` itself during the turn. The harness therefore reads the trailers in $h_0..h_1$ and credits those versions too. Before this fix, a run that grew four versions reported that it had committed none.

Termination. The loop stops on the iteration budget, on a `.avo/STOP` file, after three consecutive idle turns, or when the agent binary cannot start. The stop file needs no process to find, and the agent can create it when the task is done. The idle rule exists because "an unchanged tree records no attempt, so the stall detector cannot see it". A timed-out turn does not stop the loop; its process group is killed and the next turn starts. The supervisor never stops the loop. It only steers.

Crash safety. The run manifest is the same object as the `-json` report, and it is rewritten after every iteration: "a days-long loop gets killed eventually, and per-iteration versus at-exit is a recoverable record versus nothing". There is no `-resume`. Recovery is structural: each version is committed before the next turn, so a new `avo run` with the same prompt continues from `HEAD`. The second benchmark run in Section 8 did exactly this.

Budgets. `avo run` has an iteration budget and a per-turn timeout (default 900 s). It has no wall-clock budget and no cost budget. Cost is recorded but not enforced.

7.3 `avo supervise`: stagnation detection

The paper's supervisor is an agent. avocode's supervisor is a pure function of what the harness already records, plus a text template. It makes no LLM call. The design reason is in the source:

> "The output is a directive that **cites**. 'Try something else' is worthless — the agent already believes it is."

It detects two signals. Let $A = (a_1, \ldots, a_m)$ be the last 1,000 attempts, newest last, and b the best version.

Definition 2 (Stall)**.** Let $s = |\{a \in A : \text{head}(a) = \text{sha}(b) \vee \text{ts}(a) \geq \text{date}(b) + 1\,\text{s}\}|$, or $s = m$ if no version exists. Stall fires when $s \geq N$ and $s > 0$, with $N = 5$ by default.

Algorithm 3 SUPERVISE(N, R)

$A \leftarrow$ last 1000 attempts; $b \leftarrow$ newest version
$s \leftarrow |\{a \in A : \text{head}(a) = \text{sha}(b) \vee \text{ts}(a) \geq \text{date}(b) + 1\text{s}\}|$
$r \leftarrow$ longest suffix of A with one non-null signature
$sig \leftarrow \{\text{STALL} \mid s \geq N \wedge s > 0\} \cup \{\text{THRASH} \mid r \geq R\}$
if $sig = \emptyset$ **then return** $(\emptyset, \perp)$
end if
$V \leftarrow$ newest 3 versions; $F \leftarrow$ up to 5 memories (dead ends first, no interventions)
$U \leftarrow$ up to 3 documents in $\mathcal{K}$ that are unexplored
return $(sig, \text{RENDER}(sig, V, F, U))$ ▷ ≤ 4000 characters

Attempts are matched to b by git `HEAD` first, not by clock: the attempt that *became* b was scored before the commit and carries the parent's SHA, and git truncates commit dates to the second. An attempt that passed **f** but did not beat b still counts.

Definition 3 (Thrash)**.** Let $\text{sig}(a)$ be the failure signature of a, or $\perp$ for a passing attempt. Let r be the length of the longest suffix of A whose signatures all equal $\text{sig}(a_m) \neq \perp$. Thrash fires when $r \geq R$, with $R = 3$ by default.

The signature is the first harness error, else the scorer's first log line, normalized so that noise does not split one failure into many: lowercase; temporary paths, hex SHAs and numbers replaced by placeholders; whitespace folded; truncated to 200 characters. A pass breaks the streak. Thrash "usually means the diagnosis is wrong rather than the edit".

There is no test for a score plateau, for repeated ideas, or for similarity between a candidate and prior versions. The harness ranks measurements, not ideas. Novelty is left to the agent, which the directive gives the evidence to judge.

The directive. Only when a signal fires does the supervisor read memory and $\mathcal{K}$, so the common case stays cheap. The directive cites:

- up to three recent versions, with score, rationale and lineage file ("a variation that only re-derives one of these is not progress");
- up to five memories, dead ends before insights;
- up to three documents in $\mathcal{K}$ that no version or memory has mentioned.

A document d with significant title terms $T(d)$ counts as *unexplored* against the corpus C of all version rationales and memories when

$$2\,\big|\{u \in T(d) : u \in C\}\big| < |T(d)|. \tag{10}$$

The bar is low on purpose: a document wrongly marked explored costs one citation, while one wrongly marked unexplored "sends the agent back down a road it already travelled". Interventions are never cited. Citing them would nest old directives inside new ones, and would put every recommended document name into the corpus, so "it would erase its own best advice". Figure 4 shows a real directive from a test fixture.

```
STEERING (avo supervise)

- stall: 5 attempt(s) since v1 (16 bytes) with no committed improvement; the threshold is 5

Already committed (a variation that only re-derives one of these is not progress):
  - v1 - 16 bytes - shrink the kernel by removing five redundant bytes (lineage/v001.md)

In K and never mentioned by any version - unexplored:
  - knowledge/loop-tiling.md - Loop tiling and cache blocking
  - knowledge/register-pressure.md - Register pressure on Blackwell

Do this next:
  1. Name the direction you are about to take, and say how it differs from every line
     above. If you cannot, you do not have a new idea yet.
  2. Change one thing, then ‘avo score‘ before you change a second.
  3. ‘avo commit --why "<the direction>"‘ when it measures better. A refusal is a
     measurement: record it with ‘avo mem add‘, do not retry it.
  4. If every direction you can name is already above, do not guess: ‘avo fan --n 4‘
     explores four at once and lets f choose.
```

Figure 4: A stall directive (abridged), generated in a fixture repository with one version, five attempts that did not beat it, and two documents in $\mathcal{K}$. When thrash fires, step 1 instead says that "the same error N times means the *diagnosis* is wrong, not the edit".

Two delivery paths. In `avo run`, the directive enters the *next* turn's prompt and is recorded as an `intervention` memory. In the pi agent, an extension runs the same detector after every `avo_score` or `avo_commit` tool result and injects the directive into the *current* session as a steering message. It sends one directive per episode: an episode key built from the best version's SHA and the start of the streak makes "5 since best" and "6 since best" the same episode. Its state is rebuilt from the session branch, so a branched session that never saw a directive gets one. Both paths count the attempt log, not the session: "two drifting counters would steer one stall twice".

7.4 `avo fan`: parallel probes

`avo fan` explores N directions at once. Each probe gets its own detached git worktree at `HEAD` and its own headless agent process. The probes run on a small model (set by `AVO_PROBE_MODEL`), because "exploration is a small-model job, exploitation is a big-model job". `avo run` uses the agent's default model. All probes get the *same* prompt; diversity comes from sampling. The skill therefore asks for a prompt that names the problem, not the edit: "delete the bounds check" gives N identical diffs.

Each probe is scored in its worktree without writing to the root attempt log, so probes never trigger the supervisor. A probe is scored even if its agent failed, since "a half-finished edit passing **f** is a real result". Worktrees with no changes are removed. The manifest is written after every probe, so `-resume` reruns only unfinished probes after a crash.

Selection. Let $\nu(x) = \text{primary}(x)$ if higher is better and $-\text{primary}(x)$ otherwise. The harness computes

$$\hat{\imath} = \operatorname*{arg\,max}_{i\,:\,\text{pass}(x_i)} \nu(x_i), \qquad \hat{\imath} = \bot \text{ if no probe passes,} \tag{11}$$

with ties going to the smallest index. This is "a hint, never a decision". Promotion is manual: `avo fan -promote` j saves and applies probe j's diff to the root tree and stops. The candidate

Table 6: Headless agent templates. A custom template can be set in `.avo/config.json`.

Agent	Command	Skill discovery
pi [65]	`pi -mode json -print -approve - <prompt>`	native `.agents/skills/`; six native tools; in-session supervisor
Claude Code [4]	`claude -print -output-format stream-json -verbose -permission-mode bypassPermissions - <prompt>`	`.claude/skills` → `.agents/skills` symlink
Codex [43]	`codex exec -json -skip-git-repo-check -sandbox workspace-write - <prompt>`	skills index in `AGENTS.md`

must then go through `avo score` and `avo commit`, which re-measures it in the real tree:

$$x_{t+1} = \begin{cases} \text{promote}(x_j) & \text{if the commit rule accepts it against } x_t, \\ x_t & \text{otherwise.} \end{cases} \tag{12}$$

The skill allows at most one promotion per step, because "two probes' diffs stacked together is a candidate neither of them measured".

Recursion guards. A probe is an agent with the same skills, so it can call `avo fan` itself, and a turn of `avo run` can call `avo run`. Four guards bound this, carried in environment variables, "the only channel surviving `spawn` into an arbitrary agent binary":

1. **Depth**: refuse when the level reaches the cap (default 3); a child gets level +1.
2. **Cycle**: refuse when the prompt's hash is already in the chain of ancestors.
3. **Concurrency**: at most $\min(8, \mathit{cpus} - 2)$ probes at once.
4. **Timeout**: 900 s per probe, after which the whole process group is killed.

A guard refusal is exit 1, not 2: "a guard is a refusal, not a harness failure". avocode uses OS processes and worktrees, not in-process sub-agents, because the design notes judge that sub-agent extensions "do not work well enough".

7.5 Agent adapters

One function, `driveAgent`, runs one headless turn for both `avo run` and `avo fan`. Table 6 lists the templates. Every flag was read off the real binaries, because "a wrong flag fails silently". The prompt follows `-`, so a prompt that starts with a dash is not read as a flag. Token counts are normalized to four disjoint fields: uncached input, output, cache read and cache write. Codex reports cached input as a subset of input, so it is subtracted.

`avo install` wires the skills into a target repository by *linking*, never copying: "a copy is a fork that stops receiving fixes". It validates each skill against the Agent Skills specification first, because pi is lenient, Claude Code is strict, and a skill must load everywhere. It also writes a managed block into `AGENTS.md` with the rules and a skills index, and leaves text outside the markers alone.

For pi, an extension adds six native tools (`avo_score`, `avo_commit`, `avo_lineage`, `avo_know_query`, `avo_know_add`, `avo_fan`). They are thin wrappers over the same functions as the CLI. A refusal is a normal result, not an error. No tool takes a working directory, because "an agent that can retarget the repo can write a version into a repo nobody is watching".

Table 7: The two benchmark targets. Both use `reduce = "dominate"`.

Target	Task	`primary`	Dir.	Configs	φ
`fuzzysearch`	all (query, word) pairs with Levenshtein distance $\leq k$	mean of per-config median ms per search	$\downarrow$	2	0.03
`arcagi3`	play ARC-AGI-3 games [5] from 64×64 frames	mean fraction of levels completed	$\uparrow$	10	0.1

8 Evaluation

The evaluation asks three questions. Does the loop make real progress on a target it has not seen? Does the commit rule keep the lineage honest? And what does a turn cost? All numbers below come from transcripts committed in the repository (`evidence/`) and from the design log (`PLAN.md`).

8.1 Benchmarks

The repository ships two targets in `bench/`, chosen to be unlike each other (Table 7). `fuzzysearch` is a speed problem with a known answer, so it tests the harness. `arcagi3` is an open capability problem, so it tests the loop. A script copies a target into a fresh git repository and refuses any path inside the avocode checkout, because `avo commit` would otherwise write the loop's lineage into avocode's own history.

`fuzzysearch`. Two configs, `small` (2,500 words, 80 queries) and `large` (5,000 words, 120 queries), both with $k = 2$, are built from a seeded pseudo-lexicon so they are identical on every machine. The repetition count adapts to the candidate's speed. Spread is measured as an interquartile range, because max-minus-min showed 51% "spread" at the fast end from one garbage-collection pause. The seed program is a full dynamic-programming matrix built from nested arrays: correct and slow. The scorer's `correct` has three gates, in order:

1. **Hash gate**: `sha256sum -c` over the protected files (the reference, corpus, runner, unit tests, the scorer and the protected list itself).
2. **Unit suite**: empty inputs, $k = 0$, exact distances, Unicode, duplicates.
3. **Full-scale check**: compare the candidate with an independent reference on the exact input that is timed, and check that the candidate did not mutate its arguments.

Gate 3 matters. The end-to-end suite ships a candidate that passes every unit test but returns nothing for corpora over 1,000 words. Only gate 3 catches it ("306 pair(s) missing"). An agent that edits both the scorer and the hash file can defeat the in-repo gate, so an external `bench/init.sh -verify` audits the protected files from outside the target after each run.

Before any agent ran, six hand-written versions (the *ladder*) were replayed through `avo score` and `avo commit` to prove that the target has headroom: two-row DP, length prefilter, prefix/suffix trim, an Ukkonen band, a length-bucketed index, and a letter-set bitmask. All six committed, and the ladder went from 1719 ms to 0.931 ms (1847×) in the committed transcript. A first candidate target, matrix multiplication, was rejected at this stage: five classical optimizations gave only 1.7× in total, because the V8 JIT already does that work.

Table 8: Settings and outcomes of the two agent runs on `fuzzysearch`.

	Run 1	Run 2 (resumed)
Turns (budget)	6 (12)	3 (6)
Per-turn timeout	480 s	900 s
Wall-clock	34 min 43 s	30 min 41 s
Stopped by	watchdog	watchdog
Versions committed	4 (v1–v4)	3 (v5–v7)
Refusals	1	0
Supervisor interventions	0	0
Cost	$\geq$ \$1.91 (2 of 6 turns recoverable)	\$8.19 (\$2.73/turn)
f intact (external audit)	pass	pass

`arcagi3`. A policy plays ten offline ARC-AGI-3 games with a budget of 250 actions per episode and 24 seeded rollouts per game. Games were removed if their baseline was zero, if they were not reproducible (a real-time game scored 0.483, 0.533 and 0.483 on identical code), or if they were saturated. The scorer adds a sandbox through Python audit hooks that refuses network access and reads of the game files while the policy acts. An earlier version raised a plain `Exception`, and "a policy wrapping its body in `except Exception: pass` swallowed the violation and scored a clean 0.26". Eight held-out games and the official API are scored outside **f**, because memorization is the one thing **f** cannot catch. A scripted two-rung ladder moves the training score from 0.277 to 0.320 and the holdout from 0.217 to 0.227; its second rung is refused because one game drops by 100%. **No agent run on `arcagi3` is recorded**, so we report it as a built and gated target, not as a result.

8.2 Setup of the agent runs

Both runs used `avo run -agent claude` with Claude Code's default model on one workstation. $\mathcal{K}$ was seeded with four notes written offline: Ukkonen's banded DP, Myers' bit-parallel algorithm, BK-trees with Levenshtein automata, and V8 numeric-loop performance. The notes "deliberately do not contain the ladder". qmd and web search were not available, so $\mathcal{K}$ used the local-scan fallback. A watchdog created `.avo/STOP` after a fixed time (1,500 s for run 1, 1,200 s for run 2), so **both runs stopped on the clock, not on convergence**. Run 2 resumed the lineage of run 1 with the same prompt. Before it started, the operator removed run 1's refused candidate from the working tree and saved it as evidence. Table 8 lists the settings.

8.3 Results

Table 9 and Figure 5 give the lineage. Seven versions took the median search time from 1810.37 ms to 0.231 ms, a 7837× speedup, in nine turns and about 65 minutes of agent time. Run 1 alone reached 5255× in four versions. Every version satisfied Definition 1 against its parent with $\varphi = 3\%$, so no regression beyond the floor was committed on either config.

Reproduction. An audit script re-measured every version from its own commit in a detached worktree. All fresh measurements were slower than the recorded ones, by 0.9% to 12.9%, all in one direction, because "the box was busy". The audit's pass band is 50%, so all seven versions passed. The audit also confirmed a monotone best and an intact **f**.

Table 9: The agent lineage on `fuzzysearch`. Times are medians in ms at commit time. Speedups for v5–v7 versus v0 are computed by hand from the recorded primaries; the run transcript's "vs v0" column is wrong for run 2 (it uses the run's start as v0).

Ver.	Run	Technique (from the commit rationale)	`small`	`large`	`primary`	vs. prev	vs. v0
v0	–	full DP matrix, nested arrays (seed)	–	–	1810.37	–	1×
v1	1	rolling `Int32Array` Ukkonen band, length-bucketed corpus, flat code units	3.447	10.261	6.854	264.1×	264×
v2	1	character-set bitmask prefilter before the banded DP	0.435	0.849	0.642	10.7×	2820×
v3	1	**partition (pigeonhole) hash index** — not in the ladder	0.309	0.507	0.408	1.57×	4437×
v4	1	PassJoin position-aware probe window on that index	0.253	0.436	0.345	1.18×	5255×
v5	2	one interleaved `Int32Array` for the index	0.248	0.413	0.331	1.04×	5478×
v6	2	Myers bit-parallel for survivors; layout fused into index build	0.173	0.368	0.270	1.22×	6693×
v7	2	no flat corpus buffer; branchless Myers with early exit	0.158	0.304	0.231	1.17×	7837×

8.4 Case studies

A step outside the ladder. v1 fused five of the six ladder steps into one commit, and v2 matched the ladder's last step. v3 went further. The agent cut each corpus word into $k+1$ disjoint segments and hashed them. By the pigeonhole principle, any match within k edits keeps at least one segment intact, so the scan becomes a few hash probes. A source comment records that this cut the candidates from about 283k to about 23k on the scored workload. The commit rationale cites the bucketing note in $\mathcal{K}$: "the same idea applies to any filter whose key is discrete". v4 (PassJoin windows), v6 (Myers bit-parallel) and v7 also go beyond the ladder. The design log concludes that "the ladder is a floor on this target, not a ceiling". We note that the seeded notes named the relevant algorithm families, so $\mathcal{K}$ shaped this result.

The one refusal. In run 1, turn 6 (which hit the 480 s timeout) left a candidate that interleaved the index entries. It was slower than v4 on `small` by 4.74%, outside the 3% floor, and the rule refused it: *"'small' regressed (worst: small −4.74%) (floor ±3.00%)"*. In run 2, a fresh agent reached the same idea and made it win as v5: a tie on `small` (+2.0%, inside the floor) and a gain on `large` (+5.3%). The dead-end record did not block a better implementation of the same idea. It only blocked the version that regressed.

A wrong note, corrected by measurement. Each turn of run 2 started by measuring. The turn that produced v6 timed each phase and found that the banded DP took 30–43% of the call. Notes from v4 and v5 had called it "almost certainly too small to matter now". The agent recorded that the earlier note was wrong.

The rule shapes the step. Two of the three turns in run 2 bundled changes on purpose. A change worth +2.66% is inside the 3% floor, so it cannot commit alone, and the agent shipped it with a partner. The design log calls this "the commit rule shaping the variation step". It also conflicts with the skill's "one idea per candidate" rule, which Section 9 discusses.

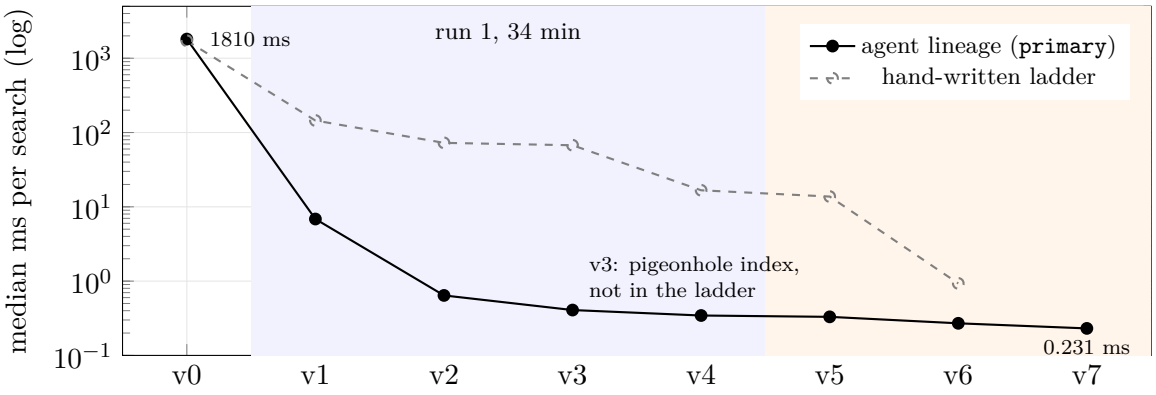


Figure 5: The `fuzzysearch` lineage on a log scale (7837×, no committed regression). The dashed line is the hand-written ladder from a separate transcript, shown for scale; its v0 is a different measurement of the same seed. The ladder's last rung (0.931 ms) is slower than the agent's v2.

A dead end found twice. The turns for v6 and v7 both tried to drop a counting sort and both measured a regression. The second turn retried on purpose, reasoning that a later change should have removed the first cause. It had not.

8.5 Cost and tokens

Run 2 used 158 uncached input tokens, 119,912 output tokens, 5,555,257 cache-read tokens and 241,350 cache-write tokens, for $8.19 or $2.73 per turn. At that rate, the default budget of 12 turns costs about $33, and `avo run` has no cost cap. Run 1's cost is only a floor. Its manifest recorded 44 of 985,039 input tokens (0.0045%) because of two bugs: the parser read only uncached input, and the output cap kept the head of the agent's stream while Claude Code writes usage and cost in the last event. Four of six logs lost that event. Both bugs are fixed, and a replay tool re-parses saved logs with the fixed parser. Bugs like this one, and an agent's own commits not being credited to its turn, appeared only in real runs; stub agents in the test suite did not expose them.

8.6 Engineering validation

The repository has 441 unit tests (`node:test`, about 4.6 s) and 14 end-to-end suites that run the real `bin/avo` against temporary git repositories with a stub agent. Each suite writes a transcript to `evidence/`; the current transcripts hold 496 passing checks. Continuous integration runs lint, type checks, the unit tests and the end-to-end suites on every pull request, with no agent installed on purpose. Examples of what the suites pin down: the stall fires at exactly N attempts and not before; a directive is appended to the turn prompt, not substituted; six worsening scores in a pi session produce *one* directive; a second `avo install` produces no diff; and a candidate that is fast but wrong at full scale fails `f`.

The supervisor fired in these tests and *never* in a real run. Across nine real turns, attempts-since-best peaked at 2. "A target with 5000x of headroom is improved faster than a stall detector can notice one."

Table 10: avocode compared with the AVO paper.

Aspect	AVO paper	avocode
Domain	CUDA attention kernels on B200	any repository with `.avo/score`
$\mathbf{f}$	vector of TFLOPS per config; 0 if incorrect	JSON line with optional vector; `null` if ¬pass, refused before ranking
Acceptance	"matches or improves" the best; vector rule and tolerance not given	Definition 1: dominance, strict gain on one config, relative floor φ; ties refused
Anti-gaming	not discussed	re-score at commit; no measuring less; frozen direction; bench hash gates
$\mathcal{P}_t$ storage	git commit with its score	commit + trailers + git note + `lineage/vNNN.md`
Failed attempts	inside the agent's context, then lost	attempt log and content-keyed dead ends, read by later sessions
$\mathcal{K}$	CUDA/PTX docs, FA4 source	`knowledge/` with provenance + the lineage itself, hybrid search
Agent	one internal agent, unnamed LLMs	pi, Claude Code or Codex via portable skills
Continuity	one conversation for 7 days	fresh process per turn; state in the repository
Supervisor	a supervisor agent; mechanism not given	deterministic stall/thrash detector; citing directive; no LLM call
Population	single lineage; branching is future work	single lineage + `avo fan` probes, one promoted
Scale of evidence	7 days, 40 versions, >500 directions	65 minutes, 7 versions, 9 turns

9 Discussion

9.1 Fidelity to AVO

Table 10 compares avocode with the paper, object by object. Where the paper is specific, avocode follows it: a single lineage, versions as git commits with scores, a vector $\mathbf{f}$ with a correctness gate, a knowledge base, and a supervisor for stall and unproductive cycles. Where the paper is silent, avocode has to choose, and it chooses the stricter option each time.

9.2 Strict acceptance and its cost

Requiring a strict gain (Equation (9)) keeps the lineage a record of progress, makes "best = newest" correct, and stops noise from committing non-changes. It has two costs that the case studies make visible.

First, a monotone lineage cannot take a worse step to reach a better region. The project's own blog post compares this to digging for diamonds and stopping just short: a promising idea whose first implementation regresses becomes a dead end. Run 2 shows that a fresh agent can reach the same idea again and implement it better, but nothing in the harness asks it to. Non-monotone acceptance in the style of simulated annealing, or an archive of promising-but-refused candidates, would address this at the price of a more complex lineage.

Second, the floor φ interacts with the "one idea per candidate" rule. A real gain smaller than φ cannot commit alone, so the agent bundles it with a partner. That makes a later refusal harder to attribute. A rule that requires both precision and small steps is in tension whenever the steps are smaller than the noise. Better measurement, for example repeated sampling of the candidate and re-measuring the incumbent on the same machine, would shrink φ and ease the tension. avocode

does neither today; it compares a fresh measurement with a stored one.

9.3 A deterministic supervisor

The paper's supervisor is drawn as an agent. avocode's is a counter, a signature matcher and a template. This is a deliberate trade. A deterministic supervisor is cheap enough to run after every turn, cannot hallucinate a citation, and is easy to test at its exact firing point. It leaves the judgment of novelty to the main agent and gives it evidence instead of advice: the versions it must not re-derive, the dead ends it must not retry, and the documents nobody has read. The cost is that it cannot recognize a repeated *idea* that fails in a new way, or a plateau of tiny committed gains. Whether this is enough is open, because the supervisor never fired in a real run.

9.4 Externalized memory as the substrate of continuity

The largest structural change from the paper is that avocode's agent is stateless. The paper's agent carries its history in context for seven days, and the paper does not say how it handles context limits. In avocode, every turn starts blank, and continuity lives in four files and a git history. This has three effects. Any agent can be the operator, including a different one on each turn. A crash loses at most one turn. And everything the loop learned can be audited, searched and reused by a later session or a human. The cost is that each turn spends tokens re-reading the past; the cache-read volume in run 2 (5.6 million tokens over three turns) reflects this.

9.5 Agent-agnosticism

Because the enforcement lives in the CLI and the instructions live in skills, the same harness runs under three agents with no per-agent logic in the commit rule. Each adapter is a short command template plus an output parser. The design forbids any capability that exists only as a pi tool, so the pi extension can only make things faster or more convenient, never different.

9.6 Self-hosting

avocode was built by a loop of the same shape. An outer loop, *ralph*, re-invoked a coding agent with a fixed prompt: orient from `PLAN.md` and the last line of `PROGRESS.jsonl`, run the health check, pick one task, prove it with a transcript in `evidence/`, commit, hand off and exit. The agent had no memory between runs. The core harness was built in one run of about three hours, and the remaining slices and four bug fixes in a second run of about four and a half hours, for a recorded cost of $129 over 18 iterations. The difference from `avo run` is the fitness signal: ralph's gate is "lint, type check and tests pass" plus a human-readable transcript, while `avo run`'s gate is a number and Definition 1. Several design points of `avo run` came from running ralph, including how it handles interrupts: an orphaned agent session once kept editing the checkout after its loop was stopped, and two agents committed to one tree.

10 Limitations and future work

10.1 Limitations of the evidence

1. **One target, one agent, one machine, two runs.** All agent results come from two `avo run` sessions with Claude Code on `fuzzysearch`. There are no repeated runs, no variance estimate, and no comparison between agents.

2. **The runs are not independent.** Run 2 resumed run 1's lineage. The 7837× figure spans both.
3. **The runs stopped on a watchdog.** The best version is a lower bound. v7 was still gaining 17% per turn.
4. **The speedup depends on the baseline.** The seed's measured time differs between sources, from 356 ms in an early design note to 1635 ms to 1810 ms in committed transcripts. The fast end agrees across sources. Ratios between versions are more stable than ratios against v0.
5. **Timing noise.** Fresh re-measurements drifted +0.9% to +12.9%, and the scripted ladder's headroom moved between 1782× and 1864× across regenerations, against a floor of 3%.
6. **The supervisor is untested in practice.** It fired zero times in nine real turns. Its default threshold of 5 comes from the design sketch, not from measurement.
7. **`avo fan` is untested in practice.** No small-model key was available, so no real run used it, and the claimed cost split between cheap probes and expensive refinement is unmeasured.
8. $\mathcal{K}$ **was seeded by hand**, with notes that name the relevant algorithm families, and qmd and web ingest were not used.
9. **The runs were supervised by a human** who set up $\mathcal{K}$, the watchdog and a `PATH` workaround, and who removed the refused candidate before run 2.
10. **No ARC-AGI-3 agent result** and no GPU kernel target. avocode reproduces AVO's harness, not its headline result.
11. **No ablations.** As in the paper, neither $\mathcal{K}$, memory nor the supervisor is ablated, and there is no comparison with a Generate $\circ$ Sample baseline at equal cost.

10.2 Limitations of the system

1. **Scorer integrity rests partly on convention.** Nothing in the harness stops an edit to `.avo/score` that inflates its numbers. The benchmark hash gates and the external audit close part of this gap, but only after the run. Locking or hashing the scorer at the harness level, and refusing any candidate whose diff touches it, is a simple next step.
2. **The incumbent is never re-measured.** A drift in the machine goes unnoticed.
3. **In-band drift.** With $\varphi > 0$, small losses inside the band can accumulate on one config over many versions.
4. **No cost or wall-clock budget** in `avo run`.
5. **`avo commit` does not re-index qmd.** New lineage pages are invisible to qmd until `avo know reindex` runs; the local fallback sees them at once.

10.3 Future work

- **Populations and archives.** Promote `avo fan` from a local probe to a population: keep promising refused candidates in an archive, in the spirit of MAP-Elites [40], and let later turns branch from them.
- **Non-monotone acceptance.** Allow bounded regressions on a side branch that must be repaid within k steps before the branch can merge into the lineage.

- **Better noise handling.** Repeated sampling with a confidence bound, and re-measuring the incumbent beside the candidate.
- **Tuning the supervisor** on a target that plateaus, and an optional LLM supervisor for comparison with the deterministic one.
- **Harness-level scorer locking** and a cost cap.
- **A GPU kernel target**, to test whether the harness can approach the paper's result with public agents, and an ARC-AGI-3 agent run scored on the holdout.
- **Ablations** of $\mathcal{K}$, memory, the supervisor and fan-out, with repeated runs.

11 Conclusion

AVO's central idea is that the variation operator of evolutionary search can be a whole coding agent rather than one LLM call. The paper showed this on attention kernels but left its harness unpublished and several of its rules unstated. avocode is an open implementation of that harness. It turns each formal object of AVO into a plain artifact in a git repository: $\mathbf{f}$ is a script that prints one JSON line, $\mathcal{P}_t$ is a chain of commits with score trailers, $\mathcal{K}$ is a search index whose corpus includes the lineage, and the agent is any of three coding agents taught by five skill files. It also makes the open rules precise. Acceptance is vector dominance with a strict gain and a noise floor, and stagnation is detected by two deterministic signals that steer with citations rather than advice.

On a fuzzy search benchmark, the loop found a 7837× speedup in about an hour of agent time, committed no regression, and found one technique that its authors had not written down. That evidence is narrow: one target, one agent, two runs, and a supervisor that never had to act. The harness is ready for the experiments that would test AVO's claim more broadly, and the most useful ones are the hardest: a target that plateaus, a population, and a kernel.

References

[1] Amirhossein Abaskohi, Mahdi Mostajabdaveh, and Zirui Zhou. SEDIMA: Cross-Run Hierarchical Insight Memory for Evolutionary Search Agents, 2026. URL https://arxiv.org/abs/2610.02361.

[2] Agent Skills. Agent skills specification. Web specification, 2025. URL https://agentskills.io/specification.

[3] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete Problems in AI Safety, 2016. URL https://arxiv.org/abs/1606.06565.

[4] Anthropic. Claude code. Software and documentation, 2025. URL https://code.claude.com/docs/en/overview.

[5] ARC Prize Foundation. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence, 2026. URL https://arxiv.org/abs/2603.24621.

[6] Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. CodeEvolve: an open source evolutionary coding agent for algorithmic discovery and optimization, 2025. URL https://arxiv.org/abs/2510.14150.

[7] Thomas Bäck, David B. Fogel, and Zbigniew Michalewicz, editors. *Handbook of Evolutionary Computation*. IOP Publishing and Oxford University Press, 1997. doi: 10.1201/9781420050387.

[8] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-Turn RL for Generating CUDA Kernels, 2025. URL https://arxiv.org/abs/2507.11948.

[9] Angelica Chen, David M. Dohan, and David R. So. EvoPrompting: Language Models for Code-Level Neural Architecture Search, 2023. URL https://arxiv.org/abs/2302.14838.

[10] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: Agentic Variation Operators for Autonomous Evolutionary Search, 2026. URL https://arxiv.org/abs/2603.24517.

[11] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cuDNN: Efficient Primitives for Deep Learning, 2014. URL https://arxiv.org/abs/1410.0759.

[12] Au Kwok Chun, Abhigyan Acherjee, Amrutha Rao, Zaiqian Chen, Kazem Meidani, C. Bayan Bruss, and Micah Goldblum. Recovering Wasted Compute in Autoresearch Agents, 2026. URL https://arxiv.org/abs/2608.10424.

[13] Weinan Dai, Hanlin Wu, Qiying Yu, Huan ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation, 2026. URL https://arxiv.org/abs/2602.24286.

[14] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL https://arxiv.org/abs/2307.08691.

[15] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022. URL https://arxiv.org/abs/2205.14135.

[16] Stefan Droste, Thomas Jansen, and Ingo Wegener. On the analysis of the (1+1) evolutionary algorithm. *Theoretical Computer Science*, 276(1–2):51–81, 2002. doi: 10.1016/S0304-3975(01)00182-7.

[17] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution, 2023. URL https://arxiv.org/abs/2309.16797.

[18] Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao, and Adam Zsolt Wagner. Mathematical exploration and discovery at scale, 2025. URL https://arxiv.org/abs/2511.02864.

[19] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers, 2023. URL https://arxiv.org/abs/2309.08532.

[20] John H. Holland. *Adaptation in Natural and Artificial Systems*. MIT Press, 1992. doi: 10.7551/mitpress/1090.001.0001. First published 1975 by University of Michigan Press.

[21] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems, 2024. URL https://arxiv.org/abs/2408.08435.

[22] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-Driven Exploration in the Space of Code, 2025. URL https://arxiv.org/abs/2502.13138.

[23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2023. URL https://arxiv.org/abs/2310.06770.

[24] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. URL https://github.com/karpathy/autoresearch.

[25] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992. ISBN 978-0-262-11170-6.

[26] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. Google DeepMind blog, 2020. URL https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/.

[27] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution, 2025. URL https://arxiv.org/abs/2509.19349.

[28] Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. The AI CUDA engineer: Agentic CUDA kernel discovery, optimization and composition. Sakana AI technical report, 2025. URL https://pub.sakana.ai/ai-cuda-engineer.

[29] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization, 2025. URL https://arxiv.org/abs/2509.14279.

[30] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[31] Joel Lehman, Jeff Clune, Dusan Misevic, Christoph Adami, Lee Altenberg, Julie Beaulieu, Peter J. Bentley, Samuel Bernard, Guillaume Beslon, David M. Bryson, et al. The Surprising Creativity of Digital Evolution: A Collection of Anecdotes from the Evolutionary Computation and Artificial Life Research Communities, 2018. URL https://arxiv.org/abs/1803.03453.

[32] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through Large Models, 2022. URL https://arxiv.org/abs/2206.08896.

[33] Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning, 2025. URL https://arxiv.org/abs/2507.14111.

[34] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, et al. SOL-ExecBench: Speed-of-Light Benchmarking for Real-World GPU Kernels Against Hardware Limits, 2026. URL https://arxiv.org/abs/2603.19173.

[35] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, 2024. URL https://arxiv.org/abs/2408.06292.

[36] Tobi Lütke. qmd: mini CLI search engine for your docs, knowledge bases, meeting notes, whatever. GitHub repository, 2025. URL https://github.com/tobi/qmd.

[37] David Manheim and Scott Garrabrant. Categorizing Variants of Goodhart's Law, 2018. URL https://arxiv.org/abs/1803.04585.

[38] METR. Recent frontier models are reward hacking. METR blog, June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking.

[39] Elliot Meyerson, Mark J. Nelson, Herbie Bradley, Adam Gaier, Arash Moradi, Amy K. Hoover, and Joel Lehman. Language model crossover: Variation through few-shot prompting. *ACM Transactions on Evolutionary Learning and Optimization*, 4(4):1–40, 2024. doi: 10.1145/3694791.

[40] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL https://arxiv.org/abs/1504.04909.

[41] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

[42] Michael O'Neill, Leonardo Vanneschi, Steven Gustafson, and Wolfgang Banzhaf. Open issues in genetic programming. *Genetic Programming and Evolvable Machines*, 11(3–4):339–363, 2010. doi: 10.1007/s10710-010-9113-2.

[43] OpenAI. Introducing Codex. OpenAI announcement, 2025. URL https://openai.com/index/introducing-codex/.

[44] OpenAI. Codex CLI: a lightweight coding agent that runs in your terminal. GitHub repository, 2025. URL https://github.com/openai/codex.

[45] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels?, 2025. URL https://arxiv.org/abs/2502.10517.

[46] Yaonan Qu and Meng Lu. Bilevel Autoresearch: Meta-Autoresearching Itself, 2026. URL https://arxiv.org/abs/2603.23420.

[47] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[48] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. ISBN 978-0-13-461099-3.

[49] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision, 2024. URL https://arxiv.org/abs/2407.08608.

[50] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. GitHub repository, 2025. URL https://github.com/algorithmicsuperintelligence/openevolve.

[51] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL https://arxiv.org/abs/2303.11366.

[52] Chunhui Wan, Xunan Dai, Zhuo Wang, Minglei Li, Yanpeng Wang, Yinan Mao, Yu Lan, and Zhiwen Xiao. LoongFlow: Directed Evolutionary Search via a Cognitive Plan-Execute-Summarize Paradigm, 2025. URL https://arxiv.org/abs/2512.24077.

[53] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL https://arxiv.org/abs/2305.16291.

[54] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents, 2024. URL https://arxiv.org/abs/2407.16741.

[55] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, 2025. URL https://arxiv.org/abs/2504.08066.

[56] Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Noveen Sachdeva, Weili Wang, Ed H. Chi, Shivaram Venkataraman, Wang-Cheng Kang, Derek Zhiyuan Cheng, and Beidou Wang. PACEvolve++: Improving Test-time Learning for Evolutionary Search Agents, 2026. URL https://arxiv.org/abs/2605.07039.

[57] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large Language Models as Optimizers, 2023. URL https://arxiv.org/abs/2309.03409.

[58] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. URL https://arxiv.org/abs/2405.15793.

[59] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models, 2022. URL https://arxiv.org/abs/2210.03629.

[60] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution, 2024. URL https://arxiv.org/abs/2402.01145.

[61] Steve Yegge. Beads: a memory upgrade for your coding agent. GitHub repository, 2025. URL https://github.com/gastownhall/beads.

[62] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement, 2024. URL https://arxiv.org/abs/2410.04444.

[63] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to Discover at Test Time, 2026. URL https://arxiv.org/abs/2601.16175.

[64] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling, 2026. URL https://arxiv.org/abs/2603.05451.

[65] Mario Zechner. pi: AI agent toolkit: unified LLM API, agent loop, TUI, coding agent CLI. GitHub repository and https://pi.dev, 2025. URL https://github.com/earendil-works/pi.

[66] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation, 2023. URL https://arxiv.org/abs/2310.02304.

[67] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents, 2025. URL https://arxiv.org/abs/2505.22954.

[68] Yifan Zhang, Yunheng Zou, Shaokun Zhang, Jian Hu, Hao Zhang, Binfeng Xu, Jan Kautz, and Yi Dong. Agora: Git as Shared Memory for Collective AutoResearch, 2026. URL https://arxiv.org/abs/2609.18094.

[69] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases, 2025. URL https://arxiv.org/abs/2510.20270.

A Command reference

Every command accepts `-json`. Exit codes are 0 for success, 1 for a refusal or a run that failed, and 2 for a harness or usage error.

B Configuration

A malformed file falls back to *all* defaults with a warning that names the field, so a typo can never disable the gate.

C A minimal session

```
git clone https://github.com/marcoscannabrava/avocode && cd avocode && ./install.sh
avo doctor                      # what is missing?

cd /your/repo
avo init                        # .avo/, lineage/, knowledge/
avo install                     # wire your agent to the skills
```

Command	Purpose	Key flags
`avo init`	scaffold `.avo/`, `knowledge/`, `lineage/`, qmd, beads	`-scorer`, `-prefix`
`avo install`	wire skills and `AGENTS.md` for agents	`-agent pi|claude|codex|all`, `-force`
`avo doctor`	report dependencies and key presence	–
`avo score`	run **f**, validate, record the attempt	`-parallel`, `-timeout`, `-init`, `-no-record`
`avo commit`	score, compare, persist only if better	`-why`, `-dry-run`, `-parallel`
`avo lineage`	list $\mathcal{P}_t$; `show <n>`; `diff <a> <b>`	–
`avo best`	the incumbent	–
`avo mem`	list; `add "<insight>"`; `prime`	`-key`
`avo know`	`init`, `query`, `add`, `search`, `reindex`	`-lexical`, `-n`, `-ingest`, `-backend`
`avo fan`	N worktree probes, scored	`-n`, `-prompt-file`, `-model`, `-promote`, `-resume`, `-clean`
`avo run`	the continuous loop	`-prompt-file`, `-max-iters`, `-agent`, `-timeout`, `-stall`, `-thrash`, `-dry-run`
`avo supervise`	stall/thrash detection and directive	`-stall`, `-thrash`

Field	Type	Default	Meaning
`reduce`	`dominate` | `mean`	`dominate`	how the vector becomes a decision
`floor`	number ≥ 0	0	symmetric relative noise band φ
`weights`	map of numbers ≥ 0	each 1	weights for `mean`
`configs`	list of names	probe	declared configs; skips the `-configs` probe
`supervise.stall`	integer ≥ 1	5	attempts since the best before a directive
`supervise.thrash`	integer ≥ 2	3	same-signature failures before a directive
`agent`	object	none	custom headless agent template

```
avo score --init hyperfine      # or pytest, vitest; then edit .avo/score
avo score --json                # check the contract by hand
avo run --prompt-file task.md --max-iters 20 --dry-run
avo run --prompt-file task.md --max-iters 20
avo lineage                     # what survived
```

D Excerpt of the `avo-vary` skill

```
# avo vary - one variation step

You are the variation operator. In the AVO formulation Vary(P_t) = Agent(P_t, K, f):
you read the population P_t (the committed lineage), consult knowledge K, propose a
change, and the fitness function f (.avo/score) judges it. You do not decide what gets
kept - avo commit does, and it only keeps a candidate that beats the current best. Your
job is to generate variation, not grade it.

## Rules that matter
- One idea per candidate. Two changes in one commit and a refusal tells you nothing
  about which one cost you.
- Never edit .avo/score to make a candidate pass. That is gaming f, and it invalidates
  every earlier version in the lineage.
```